Scalable and Efficient Simulation of LLM Training

***Abstract*—Simulation offers unique values for both enumeration and extrapolation purposes, and is becoming increasingly important for managing the massive machine learning (ML) clusters and large-scale distributed training jobs. In this paper, we build Moye to tackle three key challenges in large-scale training simulation: (1) tracing the runtime training workloads at each device in an ex-situ fashion so we can use a single device to obtain the actual execution graphs of 1K-GPU training, (2) accurately estimating the collective communication without high overheads of discrete-event based network simulation, and (3) accounting for the interference-induced computation slowdown from overlapping communication and computation kernels on the same device. Moye supports both dense and MoE workloads with high fidelity and further enables GPU memory–aware ex-situ simulation. Moye delivers on average 8% error in training step—∼3x lower than state-of-the-art simulators—for GPT-175B on a 96-GPU H800 cluster with 3D parallelism on Megatron-LM under 2 minutes.**

I. Introduction

The unprecedented success of large language models (LLMs) has been driven in large part by the large-scale infrastructures that allow the model and training dataset to scale. Organizations have constructed massive clusters with tens of thousands of GPUs to train models with hundreds of billions or even trillions of parameters [1]–[3], in a continuous stride to improve model capabilities.

Training LLMs on such massive scales consumes substantial time and financial resources. It is also inherently complex and rapidly evolving with a myriad of algorithm/model innovations [4], [5], parallelism strategies [6]–[8], kernel optimization ([9], [10]), and hardware designs [11]. As a result, accurate simulation of LLM training has started to garner attention as an essential building block to effectively manage training infrastructures and systems [12]–[16].

Broadly, simulation is useful for two main purposes: enumeration and extrapolation. In LLM training context, simulation can be used to enumerate combinations of parallelism strategies, optimization techniques, and hardware configurations that are available in the current cluster, to derive the optimal training plan for a given job and to determine efficient resource allocation schedules across jobs [12], [13]. Simulation is also useful (perhaps more so) to extrapolate beyond the currently available resources, which is paramount for strategic decision making such as capacity planning [1], [17] that involve what-if questions with significant impact. For example, what speed-up can be expected by scaling the current cluster by a factor of 5, or by increasing the network bandwidth by 2x? This also greatly facilitates the development of new optimizations, which only need to be prototyped on a small scale for the simulator to extrapolate its potential benefits on a much larger scale.

Prior work has made salient progress on training simulation [12]–[16], [18]–[21]. Yet, they fall short in three key aspects that hinder their capabilities in supporting both enumeration and extrapolation practically.

First, LLM training employs data parallelism (DP), tensor parallelism (TP), pipeline parallelism (PP), and expert parallelism (EP) for MoE models, to break up a large model along different dimensions. To accurately simulate various parallelism strategies and their memory usages on each device, one needs to obtain the training workloads, which encompass the device- or rank-specific execution graphs detailing the computation, communication, and memory events, along with their dependencies. Existing work [12], [15], [16], [19], [22] requires a full-scale cluster deployment of the job to trace the training workloads from each and every device at runtime. This is because each device builds its own execution graph independently in parallel during job initialization to most efficiently carry out training of its unique sub-model. The *in-situ* tracing approach is clearly very costly and sometimes even impractical for extrapolation usecases.

Second, simulating collective communication (CC) is critical because (1) it pieces all the devices and nodes together according to the parallelisms, and (2) its performance can often be the bottleneck [1], [23]–[25]. Most existing work [13]–[16], [19] rely on a coarse-grained α-β model [26] to simulate the CC time without considering the actual communication patterns and implementation optimizations. On the other hand, more recent work such as SimAI [12] explicitly simulates each peer-to-peer (P2P) send and receive primitive of a CC kernel using a packet-level event-based network simulator such as ns-3 [27]. This fine-grained approach provides high accuracy at the cost of prohibitive overheads even at a medium scale: simulating a 128-GPU job takes SimAI over 2 hours with an optimized ns-3 implementation (§II-C).

Third, overlapping communication and computation operations, an optimization widely used to improve efficiency [1], [28]–[30], incurs non-negligible slowdowns to the computation due to contention for shared resources (e.g., cache, DRAM bandwidth) [13], [31]. This, however, has largely been overlook thus far. Simulating interference-induced slowdown is particularly challenging since it depends on many idiosyncrasies ranging from scheduling logic of the ML framework and the underlying library to hardware-specific implementation optimizations. Much of these details are vendor-proprietary and inaccessible.

In this work, we build Moye, a practical simulation platform for large-scale LLM training. Moye is built with three key design choices. (1) It adopts an *ex-situ* tracing approach to accurately capture training workloads using just a single

device, avoiding the need for a full-scale cluster deployment. The key technical idea here is to transform the parallel initialization process at each rank into a sequential one, allowing a single device to act as each rank iteratively to trace the corresponding execution graph and GPU memory footprints. (2) It employs white-box modeling to simulate a CC kernel's performance with four parts: connection setup overhead, intra-server and inter-server transmission, and possible data reduction time. The model is supplemented with exhaustive profiling to obtain key parameters (e.g. chunk size which determines number of rounds of transmission for a message) optimized by CC libraries like NCCL for different hardware features and software configurations that affect the individual components above. This hybrid approach strikes a good balance between accuracy and efficiency. (3) It introduces an black-box ML-based slowdown predictor to model the slowdown caused by overlapping operations. The XGBoost-based predictor relies on categorical features such as transmission protocol and channel configuration in NCCL, to numerical kernel-level performance statistics such as streaming multiprocessors (SM) occupancy and DRAM utilization, to capture their complex interactions.

We implement Moye to support mainstream training frameworks: PyTorch, DeepSpeed [32], and Megatron-LM [17]. Our implementation with ∼10k LoC also includes a suite of automation tools to facilitate workload tracing and runtime profiling. We evaluate Moye on H800 and A800 clusters across a variety of scenarios and models, achieving an average prediction accuracy of 91.4% in training step time, and is up to 3x better than the state-of-the-art. For 96-GPU training of GPT-175B, Moye achieves 92% accuracy in less than 2 minutes. Beyond dense models, Moye provides high-fidelity support for MoE workloads (average error 3.4%) and further enables GPU memory–aware ex-situ simulation. We plan to open source Moye to the community.

II. Background and Motivation

A. Large-scale LLM Training

Figure 1 illustrates the layered technical stack underlying LLM training, from high-level models and frameworks to low-level GPU libraries and networked systems.

LLM training jobs are executed on frameworks such as Megatron-LM [17] and PyTorch [12], [17], [32]. They implement parallelism strategies combining DP, TP, PP, and EP to efficiently scale model training across large GPU clusters. Chiefly, DP replicates the model and splits the data, TP divides model parameters among GPUs, PP stages model layers for pipelined execution, while EP distributes experts of a MoE model across devices. Collective Communication Libraries (CCLs) then facilitate data transfer across ranks. NCCL is the de-facto CCL in production [12], [17], [32], [33].

B. Design Goals

Our overarching goal is to build a practical simulator for large-scale LLM training. Specifically, we aim to achieve:

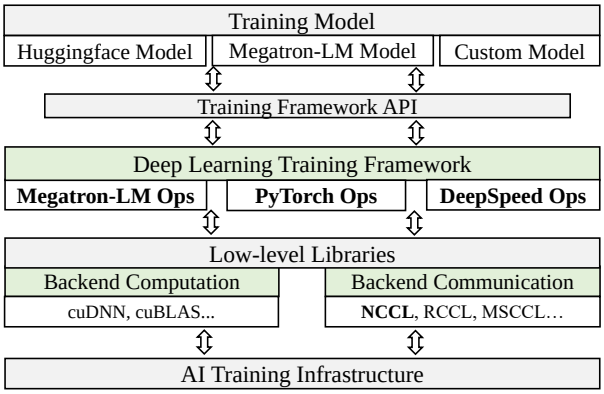

Fig. 1: Overview of the technical stack for LLM training.

- **High accuracy.** The basic goal is to accurately predict the end-to-end training step time (§VIII).
- **Ex-situ simulation.** If the simulator's input is collected by deploying the job to the target cluster at full scale, i.e. *in-situ* simulation, its utility is fundamentally limited by what is physically available. To be able to explore scales and scenarios beyond what's available, we desire *ex-situ simulation* design that can simulate a 1k-GPU cluster using a single rank for example (§IX).
- **Generality and ease of use.** The simulator should support mainstream training frameworks, model architectures, and parallelism strategies. It should also be easy to use, automating the process as much as possible to minimize manual effort (§VIII-C).
- **High efficiency.** The simulator should be fast for large-scale settings to allow efficient exploration of the vast space of training setup (§VIII-B and §IX).

C. Design Challenges

Training simulation has attracted attention recently [12]–[14], [21]. Inspired by prior work, we also separately consider computation and communication, and computation can be accurately simulated by offline profiling on the target device. Yet, to achieve the four design goals, there are three key challenges that prior work has not fully addressed. Table I summarizes existing simulators along these dimensions.

Challenge 1: *How to obtain real training workloads on each device without a full-scale deployment?*

Accurate training simulation relies on obtaining the real *training workload* as the foundation. *Training workload* refers to the execution graph on a target device that encodes the dependencies or execution sequence of computation and communication operations, plus other necessary information like tensor shapes and data types. It is the blueprint for replaying the training process on the given device. In LLM training with complex parallelisms, each device (rank) independently initializes its assigned submodel based on TP, PP, and EP settings, resulting in unique workloads per rank. However, most existing simulators (e.g., [12]–[14], [34], [35]) do not faithfully support the parallelism strategies commonly used in industrial LLM training, such as the combination of TP, PP,

Simulator	Training Workload			3D Parallelism	MoE/EP Support	Comm. Modeling	Overlap Slowdown
	Framework-Specific Ops	Ex-Situ Simulation	GPU Memory Simulation				
FlexFlow [18]	✗	✓	✗	✓	✗	$\alpha-\beta$ model	✗
Daydream [19]	✓	✗	✗	✗	✗	$\alpha-\beta$ model	✗
dPRO [16]	✓	✗	✗	✗	✗	$\alpha-\beta$ model	✗
DistSim [15]	✓	✗	✗	✓	✗	$\alpha-\beta$ model	✗
Astra-sim [14]	✗	–	✗	–	✗	$\alpha-\beta$ model	✗
Lumos [21]	✓	✗	✗	✓	✗	Outsourced	✗
TrioSim [34]	✗	✗	✗	–	✗	$\alpha-\beta$ model	✗
SimAI [12]	✓	–	✗	✓	✗	Event-driven	✗
vTrain [35]	✗	✓	✗	✓	✗	$\alpha-\beta$ model	✗
Calculon [20]	✗	✓	Analytical	✓	✗	$\alpha-\beta$ model	✗
Proteus [13]	✗	✓	Analytical	✓	✗	$\alpha-\beta$ model	–
Moye (ours)	✓	✓	Ex-Situ Profiling	✓	✓	White-box	✓

TABLE I: Comparison of Moye against existing simulators; ✓ indicates full support, ✗ no support, and – partial or conditional support.

and EP adopted in production systems like DeepSeek-V3 [8] to scale to thousands of GPUs, and as a result they cannot directly generate realistic per-rank workloads.

Thus a runtime tracing-based approach is naturally used [15], [16], [19], [22]: the per-device execution graph is extracted after each device builds its graph in parallel (when the training jobs begins). This entails a full-scale deployment of the job on the cluster, which severely limits the practical utility of the simulator in large-scale settings. Some work (Proteus [13] and Calculon [20]) choose to choreograph the per-device training workloads on their own in order to avoid this limitation. This approach is inherently imprecise since it fails to accurately capture the nuanced runtime characteristics of *framework-specific operations*—custom or highly optimized operations based on environment-specific factors (see §VIII-C). These real workloads are also crucial for accurate GPU memory simulation, while analytical memory models (e.g., [13], [20]) without them can incur up to 2.58× error in our experiments due to the framework's complex internal memory management and allocation optimizations that are difficult to capture analytically (see §VIII-C). Therefore how to faithfully capture the training workloads without a full-scale deployment becomes our first challenge.

Challenge 2: *How to accurately simulate collective communication without high overheads?*

Communication operations are critical in distributed training, yet hard to simulate as they involve complex interactions among devices and machines, and are affected by a range of network factors such as topology, latency, congestion control, etc. Existing approaches fall into two categories. Most simulators like ASTRA-sim [14], FlexFlow [18], Calculon [20], Proteus [13], and NCCL-Predictor [36] adopt a $\alpha-\beta$ model [26], which essentially calculates the running time of a communication operation as $tensor\_size/bandwidth + latency$. Figure 2 shows that the most advanced NCCL-Predictor greatly overestimates the effective bus bandwidth of `all-reduce` operations. Note NCCL-Predictor already uses additional parameters to account for the impact of hardware, communication protocols (TCP vs RDMA), and algorithms (ring vs tree for `all-reduce`), but the over-simplified $\alpha-\beta$

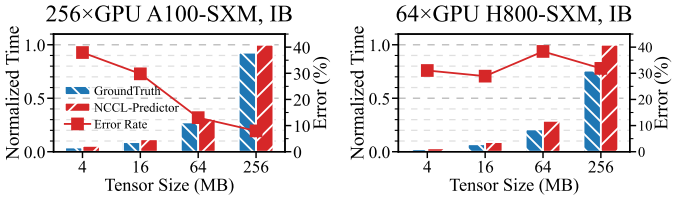

Fig. 2: Comparison of predicted communication time (by NCCL-Predictor) and ground truth for `all-reduce`.

model is limited in capturing complex interactions across all these factors. Moreover, NCCL optimizes communication by integrating hardware offloading and acceleration techniques, such as GPUDirect RDMA [37] and GDRCopy [38], which lead to variations in internal kernel behavior.

The second, more precise approach involves white-box modeling specifically targeting collective communication (CC) [12], [14]. Since NCCL (and other CCLs too) implements a CC operation as an orchestrated series of P2P send and receive operations, we can simulate each send and receive to compose the end-to-end result. Similar to the training workloads, this naturally reflects the complex optimizations used by CCLs according to topology, NCCL P2P protocol (LL, LL128 or Simple), network interconnect (IB or RoCE), algorithm (ring or tree), etc. To faithfully simulate the send/receive time, prior work [12], [14] uses packet-level discrete-event network simulators like ns-3 [27], which takes the physical topology and profiled link bandwidth as input. Yet packet-level simulation is computationally expensive as it needs to simulate the entire protocol stack at each node in the network: We run the most recent SimAI [12] with an optimized multi-threaded ns-3, and observe that simulating a single iteration on a 128-GPU cluster takes over 2 hours (7655s) on a 32-core server. Extrapolating from this measurement, a direct design-space exploration for an 8,192-GPU system over 100 training configurations would require on the order of 1.5 years of simulation time (§VIII-B), effectively confining such packet-level simulators to a small number of high-fidelity case studies and making them unsuitable for systematic configuration sweeps or rapid training-strategy search. Thus, our second challenge is how to scale white-box communication simulation efficiently so as to retain much of the accuracy of these detailed models while

Model	Overlapped op	Slowdown	Kernel Type	Slowdown
VGG19	57.53%	37.67%	GEMM	26.95%
GPT2	51.75%	10.61%	Sum	50.83%
Llama3-8B	26.79%	7.15%	GroupedGEMM	11.78%
Mixtral-8x7B	23.02%	10.55%	Grad	61.37%

TABLE II: Statistics about (1) the proportion of overlapped kernels in different models and the slowdown factors in a training step, and (2) slowdown factors of some common computation operations. Evaluated with PyTorch (VGG19), DeepSpeed (GPT2), Megatron-LM (Llama3-8B), and Flux [39] (Mixtral-8×7B).

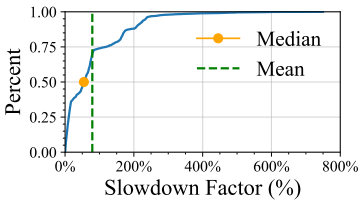

Fig. 3: CDF of slowdown factor of kernels in Mixtra-8x7B.

GPU	Kernel	CUDA	Slowdown
3090	GEMM	11.8	26.4%
	GEMM	12.1	39.3%
	LayerNorm	11.8	18.0%
	LayerNorm	12.1	93.8%
A800	GEMM	11.8	13.4%
	GEMM	12.1	17.8%
	LayerNorm	11.8	9.1%
	LayerNorm	12.1	20.7%

TABLE III: Slowdown of some kernels across GPU architectures and CUDA versions.

avoiding the prohibitive cost of packet-level simulation.

Challenge 3: *How to account for the interference between overlapping computation and communication?*

Overlapping computation and communication is widely used to improve efficiency and utilization [1], [29], [30]. We find that overlapping introduces non-negligible slowdowns especially to computation operations, even though they are assigned to separate streams on the GPU. Here, we define slowdown as the percentage increase in running time when overlapping communication kernels compared to no overlap.[1] As shown in Table II, at least 23% computation operations overlap with communication (across training frameworks), leading to a slowdown of up to 1.37x. We also examine the slowdown to individual kernels. Table II (second column group) shows when overlapped with communication kernels with 25MB messages, running times of common kernels increase by an average of 37.73%. The CDF of slowdown factors for all computation kernels of Mixtral-8x7B is presented in Figure 3, where slowdown can reach 800% with an average of 79.2%. Some recent work [31] also reported similar slowdowns in production training clusters. Despite its salient impact, overlap-induced slowdown has been overlooked by most existing work. Proteus [13] uses a simple heuristic factor that varies only with GPU architecture and ML model to account for this slowdown, which is too coarse to be accurate and not generalizable to unseen models.

Simulating the overlap-induced slowdown is a daunting task. Ideally, we need to model the kernel-level behavior in order to know exactly how the kernels overlap. For example, in the widely-used gradient overlap optimization [30], the simulator must correctly replay the backpropagation computation kernels and schedule communication kernels at precise times to achieve accurate overlap. Further, for a given computation kernel, its slowdown factor is influenced by various factors of hardware and software setup. Table III shows the slowdown factors of two kernels on different GPUs and CUDA versions vary a lot. This is because the kernel implementation and resource scheduling ultimately determines how GPU resources are shared and contended among concurrent kernels. These proprietary details are inaccessible for us, not to mention the complexity of modeling them.

[1]Experiments here are conducted on the NVIDIA A800 cluster described in §VIII-A, with software environment aligned to the configuration in §VII.

III. Design Overview

Building upon the aforementioned observations and motivations, we introduce Moye, a simulator tailored for large-scale LLM training.

Key design choices. We highlight the key choices in building Moye to address the three challenges in §II-C.

- To obtain training workloads without a full-scale deployment, Moye employs a novel ex-situ tracing approach. Generally, ML frameworks support various parallelism in the following way: (1) they initialize each rank (typically one rank per device) with its own sub-model based on the parallelism setting, and then (2) each rank builds its own execution graph to start actual training. Moye turns this parallel process into a sequential one, using only one device to act as each rank iteratively to trace the corresponding execution graph and profile the running time of each operation at the same time, achieving faithful workload tracing without requiring the full cluster.
- To achieve accurate and efficient communication simulation, Moye employs a white-box modeling approach with empirically profiled parameters that strikes a good balance between accuracy and efficiency. Our model is derived closely after NCCL's underlying chunk-based implementation of collective communication capturing critical factors such as per-chunk inter-server transmission time. These factors are profiled offline exhaustively to capture the complex dynamic optimization by NCCL according to hardware features and software configurations.
- To account for the interference between overlapping computation and communication, Moye adopts a black-box method using features related to both computation and communication kernels, such as bucket size, SM occupancy, and cache hit rate.

System architecture. Figure 4 shows Moye's architecture. Its input includes three categories: user-defined settings specifying the training framework, parallelism strategies (DP/TP/PP/EP groups); model details defining the model structure and hyperparameters; and hardware configurations detailing device specifics (GPU type), cluster size, network topology, NCCL parameters (e.g., NCCL_TOPO_FILE, NCCL_BUFFSIZE), etc. Given these inputs, the *workload tracer* extracts training workloads for each rank, including the per-rank execution graph and operation running times (§IV-A). The *CC estimator* module (§V) leverages the white-box models with parameters corresponding to the given settings to estimate each communication kernel's performance. Then the *timeline composer*

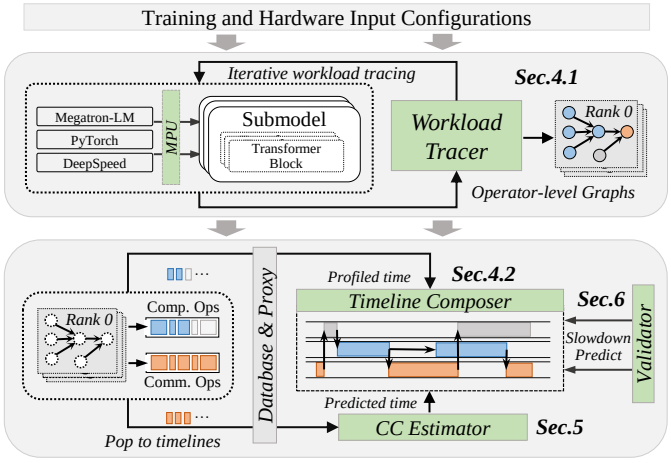

Fig. 4: Moye architecture overview. The green components represent the core modules of Moye.

module (§IV-B) re-constructs the global timelines of the end-to-end training process by assembling all per-rank execution graphs with estimated communication operation performance and inter-rank dependencies. The *validator* module (§VI) applies a machine learning model to adjust the running times of overlapping operations, accounting for the interference-induced slowdown, and outputs the final training step time result. Additionally, Moye maintains a *database* for profiling data, reducing redundant measurements across simulations.

IV. Workload Tracing

A. Per-Rank Workload Tracing

To launch a training job, the ML framework registers the global parallelism parameters (e.g. DP/TP/PP/EP groups) given by user into the so-called model parallelism unit (MPU), which maintains all states related to parallelisms for a given rank. Then according to the MPU, each rank concurrently initializes its own part of the model that it needs to train for (Figure 5 ①), and starts training with an execution graph that is optimized for its submodel based on the hardware/software environment and user configurations.

Moye extracts the per-rank execution graph in an ex-situ fashion by transforming the above parallel process into a sequential one (Figure 5 ②). Specifically, Moye hijacks all the calls to the original MPU and re-directs them to its own MPU, and registers the global parallelism parameters as usual. Moye's MPU only manipulates the input arguments to the MPU when necessary; it does not change the original implementation to ensure correctness. Then with a single device, in each iteration i, Moye uses this unique rank ID i to initialize the corresponding submodel and execute one complete training step, so all information regarding the computation operations, including their dependencies (including those w.r.t other ranks) and running times, can be profiled. For MoE workloads, Moye provides an interface to control token distributions across experts. To ensure execution isolation, Moye's tracer spawns a new process for each rank's simulation—mirroring the behavior of real-world distributed execution—thereby preventing interference such as residual CUDA memory artifacts. Based

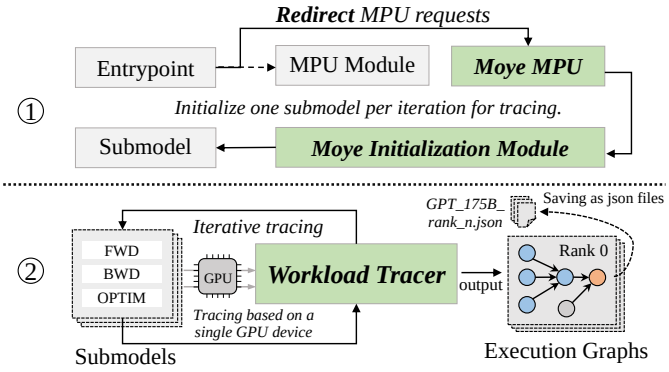

Fig. 5: Moye's MPU module and workload tracer.

on this, Moye faithfully replays the architecture's behavior on CUDA memory, including memory regions allocated for NCCL communication and optimizations such as early deallocation. This enables us to precisely and dynamically trace the memory footprint of each rank during training (see §VIII-C).

The caveat here is collective communication: one device obviously cannot execute a communication operation. To resolve this, notice that when the communication operation is launched it also needs to call into the MPU which determines the device's position in the global communication groups for proper execution. This is re-directed to Moye's MPU again allowing it to trace the communication operation's truthful information in the simulated setting (group association, message sizes, etc.); meanwhile, instead of returning results using the simulated (distributed) setting, Moye's MPU returns results corresponding to single device setting, so the communication operation can proceed and training is not blocked. This way Moye traces the actual execution graphs for each rank and running times of computation operations without requiring the full-scale cluster. Implementation details of tracing vary across frameworks which we will discuss in §VII.

Note that after per-rank workload tracing, all collective communication operations on the per-node execution graph are placeholders without any running time. The job of Moye's CC estimator and validator is precisely to estimate and refine the communication times, respectively, in the next phase.

B. Timeline Composing

Before presenting the communication estimation and validation, we discuss how Moye obtains the end-to-end training step time by composing the global timeline with the per-rank workloads obtained above.

Assuming all communication running times are known, Moye's timeline composer can recover the training process by replaying the per-rank execution graphs. timeline composer is an event-driven simulator that walks through the per-rank execution graphs and puts all operations (events) onto the global timelines accordingly, including the computation timeline, communication timeline, and memory copy (memcpy) timeline. As shown in Figure 6, a critical piece of information missing from the per-rank graphs is dependencies across ranks; for instance the downstream stages in rank i requires

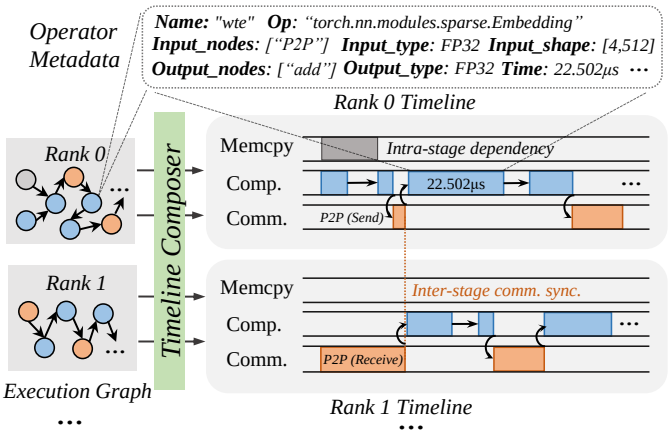

Fig. 6: Illustration of how Moye schedules operations in the execution graph to timelines across ranks.

activations from the upstream stages in rank $i-1$ before starting the forward computation in PP. We provide predefined dependencies and matching rules for both inter-stage and intra-stage events for common parallelism strategies, such as 1F1B. To enhance flexibility, this module is exposed as an API, allowing users to input new dependencies and matching rules for custom strategies.

While it is possible to strictly replay dependencies through every synchronization across all communication groups, this would significantly increase the timeline construction overhead. Instead, Moye leverages workload characteristics to selectively relax synchronization in order to accelerate simulation. For instance, in dense models, since each TP rank processes identical sequence lengths, the computation is inherently balanced, allowing us to ignore synchronization delays in TP all-reduce. For MoE models, however, load imbalance in the FFN layers requires Moye to preserve the full dependency logic to ensure correctness.

V. Communication Modeling

A. Background on NCCL

NCCL provides collective communication (CC) functions as single CUDA kernels [36] to support various training parallelisms. Our models therefore predict performance at the kernel level, a close approximation that ignores negligible launch overheads (the same applies to the slowdown prediction in §VI). A key NCCL feature is pipelining messages as smaller equal-sized chunks, which is the basis for our formulation in §V-C. For its `all-reduce` collective, NCCL supports both ring-based [36] and tree-based [40] algorithms. Other CC kernels use ring-based algorithm only.

B. White-Box Modeling: Synchronization

Moye models a NCCL kernel as two sequential stages: synchronization and execution. Upon invocation, the kernel first checks whether it can start execution based on the readiness of ranks in the current communication group. If not all ranks are ready, the kernel waits for synchronization before it can proceed. Thus, the total running time of a kernel is the sum of synchronization time and execution time. Here we investigate the synchronization time first.

#GPUs	conn_setup	data_reduction	total_trans
32	7.3%	31.7%	61.0%
64	6.4%	38.3%	55.3%
128	5.8%	46.2%	48.1%
256	5.4%	39.3%	55.4%

TABLE IV: Breakdown of running time of NCCL `all-reduce` kernel (as percentage of total time) with varying GPU counts, profiled on standard A100-SXM IB-connected clusters. The total_trans includes both intra- and inter-trans time.

Moye models synchronization time after NCCL's implementation logic. For P2P `send` and `receive`, it waits until its target communicator (i.e., each rank's communicator instance) is ready to proceed. In contrast, for CC kernels, the communicator associated with each rank in the current group must wait until all communicators have been successfully launched. The last communicator to initiate determines the actual start time of this kernel. Moye's synchronization model captures the impact of stragglers by simulating their effect on communication delays.

C. White-Box Modeling: Execution

Execution time represents the majority of kernel running time, and is the focus of our modeling. We build white-box models that closely follow the kernel execution process in NCCL under different algorithms, and profile all key parameters of the models exhaustively in various hardware/software settings to achieve accurate prediction without the high overheads of event-based simulation.

Basic model. We perform a running time breakdown analysis of CC kernel in Table IV. By inspecting NCCL's GPU-side implementation [41], [42] and tracing the control flow of CC kernels, we find that the execution time of a CC kernel can be decomposed into (at most) four *non-overlapping* phases, using `all-reduce` as an example:

$$\begin{aligned} T_{comm\_kernel} = & T_{conn\_setup} + T_{intra\_trans} \\ & + T_{data\_reduction} + T_{inter\_trans} \end{aligned} \tag{1}$$

Here $T_{\text{conn\_setup}}$ is connection setup time; $T_{\text{intra\_trans}}$, $T_{\text{inter\_trans}}$ are the times to transmit tensor chunks according to the specific algorithm, and $T_{\text{data\_reduction}}$ is the computation time of the reduce operation over the received tensor chunks and the local tensor. Note that not all four components are present in other CC kernels. Thus, this decomposition is implementation-faithful rather than an empirical regression artifact.

Building on Eq. (1) and following the per-round send/recv schedule encoded in NCCL's ring and tree kernels [41]–[43], we derive the following execution-time models for common

CC kernels:

$$T^{Ring}_{all-gather} = \alpha + \gamma \cdot \eta(N-1), \quad (2)$$

$$T^{Ring}_{reduce-scatter} = \alpha + \eta\big[(N-1) + \gamma \times (N-1)\big] + \delta \times tensor\_size, \quad (3)$$

$$T^{Ring}_{\texttt{all-reduce}} = \alpha + \eta\big[(N-1) + \gamma \times 2(N-1)\big] + \delta \times tensor\_size, \quad (4)$$

$$T^{Tree}_{\texttt{all-reduce}} = \alpha + \gamma(\eta-1) + 2\beta(K-1) + 2\gamma\log_2(M) + \delta \times tensor\_size. \quad (5)$$

Here N is the total number of devices, M the number of nodes (servers), and $K = N/M$ the number of devices per node. We denote the connection setup time as α, and the intra-server and inter-server transmission times as β and γ. The reduce time depends on *tensor_size* and the reduce throughput δ. Finally, η is the number of rounds (or chunks) for this tensor, which corresponds to NCCL's chunk-based implementation.

Our white-box framework naturally extends to the `all-to-all` collective. Instead of using dedicated ring or tree algorithms, NCCL implements `all-to-all` by decomposing it into N concurrent point-to-point `send` and `receive` pairs [41]. These are executed by the `sendrecv` kernel using chunk-based pipelining. We model its execution time as:

$$T^{P2P}_{\texttt{all-to-all}} = \alpha + \max\big(\beta \cdot \eta(K-1),\ \gamma \cdot \eta(N-K)\big), \quad (6)$$

where the max operator reflects the concurrent utilization of independent physical channels for intra-node (to $K-1$ GPUs) and inter-node (to $N-K$ GPUs) transfers. The δ term is absent since `all-to-all` involves no reduction.

This modeling decouples scale-dependent terms from fabric-dependent parameters: job scale and parallelism enter only through N, M, K, *tensor_size*, and η, while $(\alpha, \beta, \gamma, \delta)$ summarize the steady-state behavior of intra- and inter-node communication for a fixed hardware/software stack. Once these parameters are calibrated on a moderate-size cluster, the same closed-form expressions can be evaluated for much larger N and M without additional network-level simulation.

The above models treat γ as a per-hop inter-server transmission time that is primarily determined by the underlying hardware, software, and NCCL configuration. In practice, large jobs may incur additional latency from multi-tier topologies and aggregate flow-level contention that are not constant across scales. To capture these second-order effects without resorting to packet-level simulation, Moye introduces a dimensionless scale correction factor $c_{\text{scale}}(N)$ and defines an effective inter-server transmission time

$$\gamma_{\text{eff}}(N) = \gamma \cdot c_{\text{scale}}(N), \quad (7)$$

where $c_{\text{scale}}(N) \geq 1$ aggregates residual scale-dependent network effects at job size N; its dependence on the NCCL algorithm and cluster topology is left implicit for brevity. During prediction, Moye evaluates equations by substituting $\gamma_{\text{eff}}(N)$ for γ, so that the analytic dependence on N, M, and K captures the dominant algorithmic scaling behavior, while $c_{\text{scale}}(N)$ adjusts for large-scale contention.

Offline exhaustive profiling. The modeling above is explicitly related to nine parameters and implicitly to an additional parameter, *chunk_size*. Among them, N, M, K, and *tensor_size* are user configurations, while α, β, γ, δ, and η are what Moye attempts to obtain. Modeling them analytically is challenging. First, they depend on *chunk_size* and *tensor_size*. The chunk size is not a predefined constant but dynamically tuned by NCCL [36]. Further, these parameters vary with hardware configuration and software implementation (e.g., InfiniBand vs. NVLink, NCCL version). These idiosyncrasies also differ across CCLs, making a single closed-form model difficult.

We therefore adopt offline profiling to obtain these parameters, as they are fixed once the hardware/software configurations are settled. First, we modify NCCL to obtain the chunk size during the initialization phase. We enumerate representative combinations of N, M, *tensor_size*, and hardware type (InfiniBand, TCP, NVLink) to record the corresponding *chunk_size* and number of rounds η computed by NCCL. Then, for each *chunk_size*, we profile the corresponding connection setup time, intra-server and inter-server transmission time, and reduce throughput $(\alpha, \beta, \gamma, \delta)$ under the corresponding setting. This profiling requires access to a real cluster, since the effective network bandwidth can differ across 2-node, 4-node, and 8-node deployments. In common Clos-based topologies, we observe that the per-node bandwidth—and thus the baseline parameters $(\alpha, \beta, \gamma, \delta)$—quickly converge once the job spans a small number of nodes; residual scale-dependent effects beyond this regime are modeled through the correction factor $c_{\text{scale}}(N)$ (see §VII).

Taken together, combining white-box analytical models with offline-profiled parameters and a lightweight scale correction factor provides a good first-order estimation of CC kernel performance with very low overheads.

VI. Overlapping Slowdown Prediction

Accurately capturing and modeling the resource contention between overlapping kernels presents substantial challenges in predicting the slowdown factor as discussed in §II-C. To address this, Moye introduces an ML-based slowdown prediction model with a number of hardware- and software-specific features, which we present here.

A GPU devices executes concurrent computation and communication kernels with spatial multitasking, where each kernel is assigned to separate streams with a subset of the SMs. At the same time they also contend for shared resources such as cache and DRAM bandwidth. These factors results in performance interference, and Moye needs to capture them in predicting the slowdown.

Feature extraction. Table V summarizes the key features used by our model. We categorize features into two groups: communication-related details and computation kernel metrics. For communication, we consider features such as protocol and algorithm employed by NCCL, CC kernel type, bucket size, and channel configuration. These parameters capture the

Type	Feature	Specification
Comm. details	Protocol	Communication protocol used within NCCL
	Algorithm	Communication topology algorithm applied in NCCL
	Collective	Name of the collective function in the kernel
	Bucket size	Size of the tensor bucket used in collective communication
	Channel number	Number of communication channels in NCCL
Comp. Kernel Metric	Running time	Running time profiled on single-device training
	Compute throughput	Average SM throughput
	Memory throughput	Percentage of cycles where DRAM was active
	DRAM throughput	Peak DRAM throughput
	Achieved occupancy	Average percentage of active warps on each SM
	L1 hit rate	Hit rate for L1 cache
	L2 hit rate	Hit rate for L2 cache

TABLE V: Features in the slowdown prediction model.

complexity and overheads of communication: bucket size, for instance, determines when a communication kernel will be launched after accumulating one bucket of data. On the computation side, we profile kernels at a fine-grained level on a single device to measure their baseline performance without overlapping. We collect running times, compute throughput, memory utilization, and cache statistics using NVIDIA Nsight Systems and Nsight Compute. For instance, achieved occupancy and SM activity levels indicate compute intensity, while DRAM utilization and memory throughput—along with L1/L2 hit rates—shed light on memory access latency and potential bottlenecks. Additionally, we classify kernels by their functional types (e.g., matrix multiplication, layer normalization) to further refine our predictions.

XGBoost-based model. We leverage XGBoost [44], a tree-based gradient boosting framework, to predict the slowdown factors caused by overlapping kernels with the above features. XGBoost is well-suited to this task for two key reasons. First, our dataset contains heterogeneous features that are both numerical (e.g., SM throughput, DRAM utilization) and categorical (e.g., GPU type, kernel type), a scenario where tree-based models excel. Second, our target slowdown factors, influenced by the interplay of diverse system conditions, remain bounded within a range well-captured by the training set. As we expand our dataset—incorporating additional profiles from various models, GPU architectures, and cluster scales—the model's generalization ability improves, enabling accurate slowdown predictions across increasingly complex and heterogeneous environments.

VII. Implementation

We implement Moye as illustrated in Figure 4. Moye is developed based on PyTorch 2.1, DeepSpeed 0.13.1, and Megatron-LM (commit ID: 53a350ed), and NCCL 2.18.6, all running on CUDA 12.1, comprising ∼10K lines of code (LoC). ∼2K LoC are dedicated to Megatron-LM, 3K LoC to DeepSpeed and PyTorch, and the remaining 5K LoC for common core modules that underpin the simulation engine, training workload tracing, overlapping induced slowdown dataset collection, and CC parameter exhaustive profiling.

Workload tracer. We implement two tracing modules to support execution graph extraction for Huggingface models used by PyTorch and DeepSpeed and Megatron-LM models. For the former, the tracer leverages `torch.fx` to record operations. Since `torch.fx` only captures computation operation during the forward pass, we enhance it to capture operations during backward passes by using tensor gradient functions and registering hooks, as well as optimizer steps and communication operations (handled as placeholders). For Megatron-LM models, a custom tracing module is developed with a series of Python decorators and context managers. The generated execution graph is output as a JSON file and stored in a database for future use.

Model	Layers	Hidden	Heads	FFN	Experts	Expert FFN	Top-k	Attn.
Mixtral-8×1.75B	8	4096	32	–	8	14336	2	GQA
Qwen3-A3B	48	2048	32	6144	128	768	8	GQA
DeepSeek-V3 variant	32	2048	64	4096	32	512	2	MLA

TABLE VI: Key configuration parameters for the three evaluated MoE workloads. FFN denotes the dense (non-expert) FFN hidden size; Mixtral routes all FFN computation through experts.

CC estimator. To profile the key parameters for models in §V-C, we utilize the NCCL profiling tool NPKit [45]. We empirically profile these parameters on our clusters up to 4 nodes (32 GPUs) to obtain the baseline per-hop costs and additionally run NCCL-test on larger 720-GPU A100-SXM IB-connected clusters to calibrate the scale correction factor $c_{\text{scale}}(N)$ introduced in §V-C. Moye maintains a parameter database. Each entry stores the baseline parameters as well as the corresponding samples of $c_{\text{scale}}(N)$ for different job sizes. To better support a small number of high-fidelity studies, Moye also exposes interfaces for approaches such as ns-3.

Validator/Slowdown predictor. We experimentally find the best hyperparameters for our XGBoost model (max depth 12, trained on ∼8000 kernels). We implement an automated pipeline to construct the training dataset. NVIDIA Nsight Compute [46] is used to profile GPU resource metrics such as SM throughput, memory bandwidth, and cache hit rates. Nsight Systems [47] capture kernel execution traces under overlapping and non-overlapping conditions, exporting the data as SQLite databases.

VIII. Evaluation

In this section, we first evaluate Moye's end-to-end accuracy and runtime cost (§VIII-B), then break down the accuracy and effectiveness of its key modules (§VIII-C, §VIII-D, §VIII-E). We provide the experimental setup in §VIII-A.

A. Experiment Setup

Models. We evaluate Moye on both dense and MoE models. For dense workloads, we use VGG19 [48] and GPT models [17] ranging from 13B to 485B parameters. For MoE workloads, we use the Mixtral series [49], Qwen3-A3B [50], and a DeepSeek-V3 variant derived from DeepSeek-V3 [51]; their key configurations are summarized in Table VI.

Clusters. Our evaluation utilizes a 256-GPU H800 cluster (32 servers with 8x 80GB GPUs each) and an 8-GPU A800 cluster. Both use 400 GB/s NVLink for intra-server communication, while the H800 cluster is equipped with 400 Gb/s NDR InfiniBand for inter-server links. Additionally, an RTX 3090 cluster is used for kernel slowdown analysis (§VIII-E).

Baselines. We compare Moye with five fully open-source simulators:

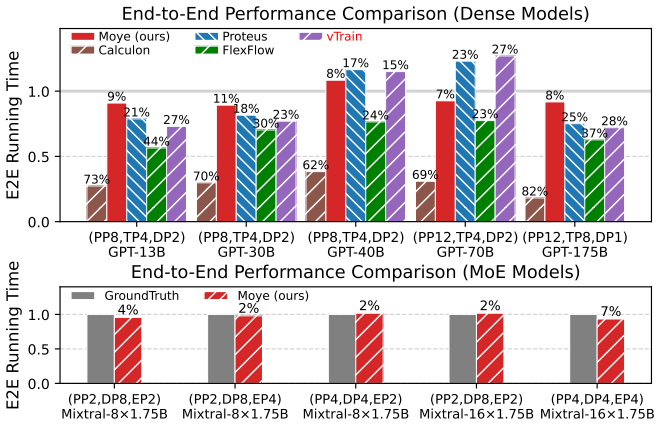

Fig. 7: End-to-end training step time comparison on dense and MoE models (up to 96 H800 GPUs). All results are normalized to ground truth (y=1.0/gray bars). Numbers above bars denote prediction error. Baseline systems do not support MoE simulation.

- **Proteus** [13]: A SOTA simulator for integrated simulation of parallel strategies and runtime behaviors.
- **FlexFlow** [18]: An internal simulator for throughput estimation across different parallelization strategies.
- **Calculon** [20]: NVIDIA's analytical performance simulator for hardware-software codesign exploration.
- **vTrain** [35]: A GPT-specific simulation framework for 3D parallel LLM training.
- **SimAI** [12]: A training simulator that employs packet-level network simulation for high-fidelity communication modeling.

ASTRA-sim [14] and TrioSim [34] are open source as well, but they suffer from critical limitations: ASTRA-sim natively supports only a narrow set of compute kernels (mainly GEMM and a few LLaMA-specific operators such as attention and MLP). TrioSim [34]'s communication predictor only supports single-node scenarios, and it lacks support for hybrid parallelism (i.e., it can only model one parallelism strategy at a time).

B. End-to-End Results

Accuracy. We evaluate the end-to-end accuracy of Moye on H800 clusters of varying scales using Megatron-LM. As shown in Figure 7, Moye consistently achieves high fidelity across diverse parallelism configurations. For 96-GPU scenarios, the prediction error is 7% for GPT-70B ($12 \times 4 \times 2$) and 8% for GPT-175B ($12 \times 8 \times 1$). In contrast, baseline simulators deviate significantly: Proteus and FlexFlow show errors exceeding 25%, while Calculon is overly optimistic (error >69%) due to its reliance on ideal analytical models. Moye also attains high accuracy on MoE models, with an average error of 3.4%. Across models, Moye achieves average errors of 4.5% and 17.8% for computation and communication operations. Its accuracy is further improved by modeling slowdown from overlapping kernels. In our settings here, computation–communication overlap accounts for 6.0%–9.8% of total step time, and simulating the overlap-induced

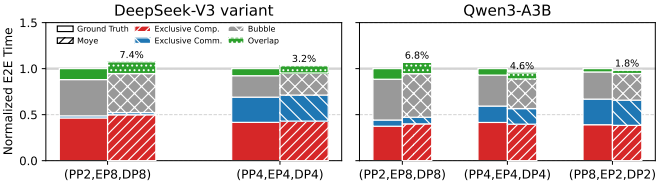

Fig. 8: End-to-end component breakdown for two latest MoE models in our H800 setup. Left: DeepSeek-V3 variant. Right: Qwen3-A3B. Each configuration shows a textured ground-truth bar and a colored stacked Moye bar decomposed into exclusive computation, exclusive communication, bubble, and overlap. Numbers above Moye bars denote absolute end-to-end prediction error. Proteus, FlexFlow, and Calculon are omitted because they do not support these latest models.

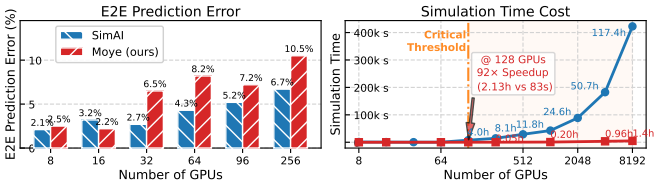

Fig. 9: Comparison of Moye and SimAI on GPT-175B (H800 cluster, 3D parallelism). Left: end-to-end prediction error across cluster scales. Right: simulation time cost (log-log scale). Beyond 128 GPUs, Moye achieves a 91.8× speedup over SimAI while maintaining comparable accuracy.

slowdown improves end-to-end accuracy by 3.6%–5.2%. As training systems increasingly optimize for aggressive overlap [52], accurate slowdown prediction may become even more critical.

Latest MoE models. We further evaluate Moye on two recently released MoE models, DeepSeek-V3 variant and Qwen3-A3B, in our H800 setup. As shown in Figure 8, other SOTA simulators cannot simulate these latest models, so we only compare Moye against ground truth. Across the five feasible configurations, Moye achieves an average end-to-end prediction error of 4.8%, with a maximum error of 7.4%. For Qwen3-A3B, the error ranges from 1.8% to 6.8%; for DeepSeek-V3 variant, it ranges from 3.2% to 7.4%. The overlap portion spans 3.5%–12.8% of total step time, while bubble remains a major contributor at 24.0%–47.3%, further highlighting the importance of modeling both pipeline idle time (via explicit dependency declarations) and computation–communication overlap when baselines are unavailable for these latest models.

Comparison with packet-level simulator. We compare Moye against SimAI [12], a packet-level network simulator built on ns-3, on GPT-175B training across 8–8,192 H800 GPUs (Figure 9). In terms of accuracy, both simulators achieve comparable end-to-end prediction errors at small scales (2–3% at 8–32 GPUs). As the cluster grows, Moye's error increases moderately to 10.5% at 256 GPUs, while SimAI remains at 6.7%—a gap attributable to SimAI's finer-grained packet-level communication modeling. However, the simulation time gap is dramatic. At 128 GPUs, Moye completes in 83 seconds—a 91.8× speedup over SimAI's 7,655 seconds. This advantage widens further at scale: at 8,192 GPUs, Moye finishes in 1.4 hours while SimAI requires over 117 hours. For a design-

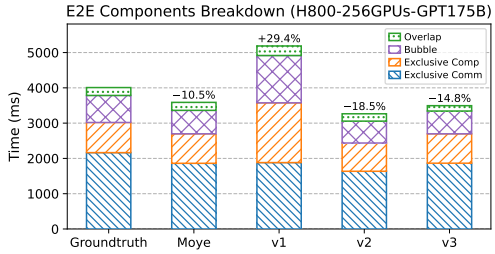

Fig. 10: Component breakdown of GPT-175B training step time on 256 H800 GPUs with 3D parallelism. Each bar decomposes the step time into exclusive computation, exclusive communication, bubble, and overlap. Numbers above bars denote end-to-end prediction error.

Variant	Excl. Comp.	Excl. Comm.	Bubble	Overlap	End-to-End
Moye (Full)	−2.2%	−14.0%	−13.2%	+1.1%	−10.5%
w/o Ex-situ Tracer (v1)	+98.2%	−13.0%	+75.0%	+21.1%	+29.4%
w/o White-box Comm. (v2)	−6.3%	−24.4%	−19.0%	−6.8%	−18.5%
w/o Slowdown Pred. (v3)	−2.5%	−13.9%	−15.7%	−31.9%	−14.8%

TABLE VII: Ablation study on GPT-175B (256 H800 GPUs, 3D parallelism). Each column shows the signed prediction error of the corresponding time component relative to ground truth.

space exploration sweeping 100 configurations on an 8,192-GPU cluster, Moye would complete in under 6 days, whereas SimAI would require over 1.5 years. These results demonstrate that Moye offers a compelling accuracy–efficiency trade-off, making it practical for iterative what-if analysis at datacenter scale where packet-level simulation is prohibitively expensive.

Ablation study. To isolate the contribution of each key module, we perform an ablation study on GPT-175B (FP16) trained on a 256-GPU H800 cluster with 3D parallelism ($\text{PP} = 16, \text{TP} = 8, \text{DP} = 2$). We disable one module at a time and report the signed prediction error for each time component against ground truth (Figure 10 and Table VII).

Removing the ex-situ computation profiler and substituting analytical FLOPS estimates inflates the exclusive computation error to $+98.2\%$, which cascades into $+75.0\%$ bubble error and $+29.4\%$ end-to-end deviation—the largest among all variants. Replacing the white-box communication model with an α-β model increases the exclusive communication error from 14.0% to 24.4% and worsens end-to-end accuracy from 10.5% to 18.5%. Disabling slowdown prediction degrades overlap estimation from $+1.1\%$ to -31.9% error, raising the end-to-end error by 4.3 percentage points. These results confirm that all three modules contribute meaningfully to Moye's end-to-end fidelity.

Simulation cost. We investigate Moye's two time costs: (1) profiling time cost, and (2) simulation time cost, including initialization and execution, and excludes the one-time, reusable workload tracing cost. As detailed in Table **??**, Moye is highly efficient across training scenarios. For DDP, simulation is consistently fast, averaging about 10 seconds. The advantage is more pronounced for complex 3D parallelism; by leveraging its white-box communication model, Moye simulates a 128-GPU setup in 83 seconds—a 91.8x speedup over packet-level simulators like SimAI (7655s). This efficiency scales to large configurations, enabling the simulation of one step of an

#GPUs	VGG19				GPT 13B∼175B			
	Init.	Execution	Sim. Total	Trace	Init.	Execution	Sim. Total	Trace
256	0.4	10.3	10.7	18.2	8.4	159.2	167.6	1350.2
1024	0.3	9.7	10.0	18.5	35.9	682.8	718.7	1926.7
4096	0.4	10.2	10.6	17.8	149.9	3307.8	3457.7	2688.3
8192	0.3	10.3	10.5	18.3	374.7	4602.2	4976.9	3498.2

TABLE VIII: The simulation time costs of Moye in seconds. VGG19 is trained using the DDP mode, while GPT models (ranging from 13B to 175B) are trained using 3D parallelism.

8,192-GPU cluster in $<$1.4 hours. For a typical design-space exploration over 100 training configurations on such a system, Moye would complete all simulations in under 6 days. At this scale, the much higher per-configuration cost of packet-level network simulation (like SimAI) would push turnaround time to 1.5 years.

C. Computation and Memory Usage Simulation

We evaluate Moye's accuracy in simulating computation workloads on both H800 and A800 clusters, using VGG19 (with DeepSpeed), GPT-13B and Mixtral-8×1.75B (both with Megatron-LM) under various PP/EP configurations.

Accuracy. Moye reconstructs the computation workload by tracing all computational operations and reproducing their execution sequence based on the execution graph. Per-GPU operation running times are derived through Moye's workload tracer without requiring a full-scale deployment. As shown in Figure 11, Moye achieves a maximum overall error of 8.31%. In contrast, while Proteus and FlexFlow maintain relatively low error rates for VGG19 under DeepSpeed (8.6% and 17.81%, respectively), their errors escalate dramatically for GPT-13B under Megatron-LM, reaching 91.53% and 109.14%, respectively. The root cause of this discrepancy is that Proteus and FlexFlow rely heavily on standard PyTorch operations, failing to incorporate the custom fused operations that frameworks like Megatron-LM employ for performance optimization. Such specialized operations cannot be accurately modeled without extracting their runtime characteristics directly from the target training framework (as we disccusion in §II). Calculon overestimates computation time in an overly idealized manner (error rate $> 400\%$), because its modeling relies on hardware's analytical FLOPS values, which severely deviate from real hardware execution. By precisely tracing, Moye captures the actual operation behaviors and achieves substantially higher accuracy.

Generality. Moye is the only high-fidelity simulator that supports MoE workloads (error $< 6\%$, Figure 11). None of the baselines can handle MoE models. Even for CNNs, Proteus and FlexFlow fail on certain configurations (e.g., VGG19 at PP=4) due to their reliance on manual, pre-hardcoded workload construction. In contrast, Moye supports simulations across a wide range of training design spaces. We further demonstrate its scalability via an ex-situ simulation of 8,192 GPUs in §IX.

GPU Memory usage. Moye supports high-fidelity GPU memory usage simulation through ex-situ profiling. Figure 12 (upper) shows the peak memory usage (including both static and dynamic components), where Moye achieves nearly 100%

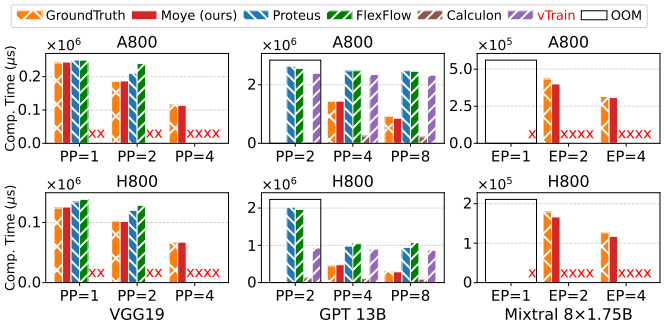

Fig. 11: Computation times of various workloads on A800 and H800 clusters under different parallelization settings. Crosses denote cases not supported by the baselines, while squares mark configurations that encountered OOM errors during real execution.

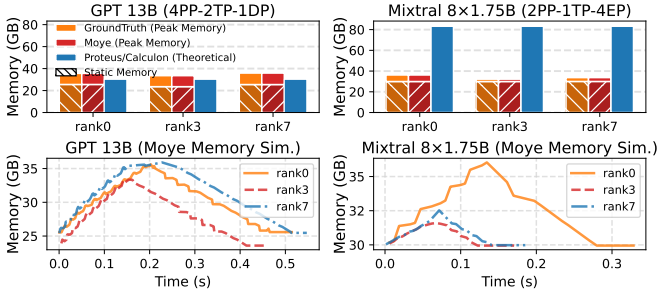

Fig. 12: Validation of Moye's GPU memory simulation for GPT-13B and Mixtral-8×1.75B on our 8-GPU A800 cluster (8 ranks), showing three representative ranks. It demonstrates Moye's ability to capture dynamic per-rank memory usage within a training step (excluding communication time). We set num_micro_batch=micro_batch_size=1 here.

accuracy on GPT-13B and Mixtral-8×1.75B for different ranks. In contrast, analytical methods can result in 18.0% error for GPT-13B, and for MoE models (Mixtral-8×1.75B), the error even increase to 2.58× (a gap of 51 GB). Moye can also reflect dynamic GPU allocation behavior during training, as shown in Figure 12 (lower)—a capability absent in existing simulators. We believe this ability can help engineers identify potential optimization opportunities.

D. Communication Simulation

We now evaluate the effectiveness of Moye's communication simulation, comparing it against NCCL-Predictor [36], the underlying communication model used by both Proteus and FlexFlow. NCCL-Predictor leverages parameters tuned from production experience and employs an $\alpha - \beta$ model (§II-C).

Accuracy. Figures 13 show the prediction accuracy of collective communication across message sizes (4MB-256MB) and cluster scales. Moye consistently outperforms NCCL-Predictor, with average errors of 12.6% and 13.8% on 8- and 12-server H800 clusters, improving upon NCCL-Predictor by 29.6% and 19.4%, respectively. Here, for 8- and 12-server clusters we set the scaling factor $c_{\text{scale}}(N)$ to 1.023 and 1.063, respectively. The gap is smallest on ring-based `all-reduce` (4.2%), where NCCL's heavy optimizations align well with the $\alpha - \beta$ model. For other patterns—especially tree-based `all-reduce`—Moye achieves much higher accuracy, reducing errors by up to 33.5%.

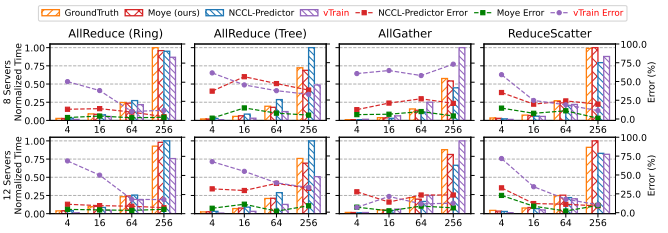

Fig. 13: Running time and prediction error for inter-server CC kernels on H800 GPUs across varying tensor sizes and cluster scales. Each subplot is normalized relative to the operation with the longest running time.

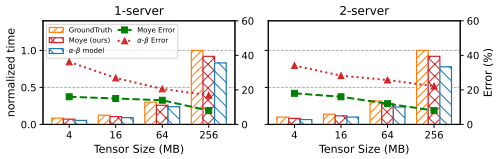

Fig. 14: Moye's all-to-all running time prediction on H800 clusters.

These improvements stem from Moye's profiling-based white-box model, which captures bandwidth utilization and synchronization overheads that NCCL-Predictor's simplified assumptions overlook. As a result, Moye maintains consistent accuracy for both small and large messages, whereas NCCL-Predictor suffers average errors of 27.6% on 4MB and 16MB inputs. Finally, Moye's white-box design also accelerates simulation speed, as shown in §VIII-B. We further evaluate Moye on the `all-to-all`. As shown in Figure 14, Moye achieves an average error of 13.6% across 1- and 2-server H800 configurations, compared to 25.8% for the α-β model.

Breakdown analysis. Table IX reports the breakdown of `all-reduce` runtime on a 32-GPU H800 cluster, separating intra-server latency, inter-server latency, and reduction computation across tensor sizes. Intra-server round-trip time increases moderately with tensor size (57.4 μs at 4MB to 99.2 μs at 256MB), reflecting the overhead of scheduling more thread blocks for larger chunks. Inter-server round-trip time shows a similar trend, growing to 279 μs at 256MB. Reduction computation dominates at large tensors, scaling nearly linearly from 50.1 μs (4MB) to 1658 μs (256MB). These results demonstrate that Moye accurately captures both communication and computation overheads across varying message sizes.

E. Slowdown Prediction

Most simulators overlook the performance slowdown caused by kernel overlap. We compare Moye with Proteus, the only baseline known to model this effect, which does so using a simple heuristic based on GPU architecture and model type.

Prediction accuracy. We evaluate slowdown prediction on ∼10,000 kernels (Table X). Moye consistently outperforms the baseline heuristic, especially on data-center GPUs. On the H800, 57.7% of predictions fall within 15% error, compared to 40.2% for the baseline, and the RMSE is nearly 3× lower (0.56 vs. 1.55). Similar gains hold on the A800, while on consumer-grade RTX 3090 GPUs, Moye again shows substantially lower error across all metrics.

Tensor size (MB)	4	8	16	32	64	128	256
Chunk size (10^3 bytes)	18	36	72	72	72	144	288
Intra-server Round trip (μs)	57.43	64.36	73.62	74.61	77.46	75.55	99.23
Inter-server Round trip (μs)	54.3	72.32	106.73	112.9	111.25	174.2	278.98
Reduce operation (μs)	50.14	65.86	102.46	219.89	463.43	841.71	1657.72

TABLE IX: The breakdown of Moye's `all-reduce` running time, profiled in the H800 cluster using 32 GPUs.

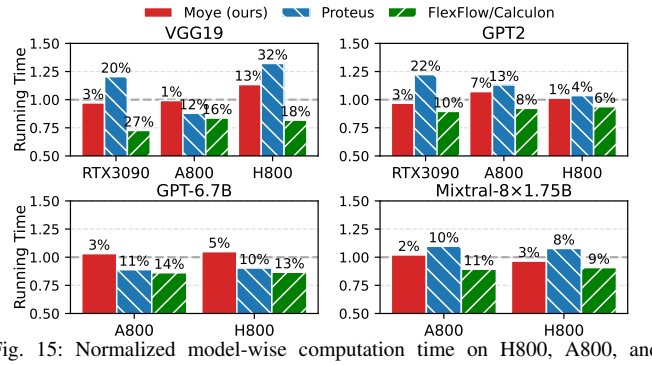

Fig. 15: Normalized model-wise computation time on H800, A800, and RTX3090 GPUs. Moye (prediction-based) is compared against Proteus (fixed-parameter slowdown modeling) and FlexFlow/Calculon (no slowdown modeling). Numbers above bars denote prediction error relative to ground truth (gray line at y=1.0).

Solution	GPU	5% Error	10% Error	15% Error	RMSE
Moye	3090	61.48%	73.62%	76.52%	1.0935
Proteus		8.52%	13.62%	19.28%	1.4953
Moye	A800	61.03%	69.82%	75.01%	0.7147
Proteus		59.04%	65.45%	70.51%	1.5170
Moye	H800	43.18%	51.59%	57.73%	0.5626
Proteus		34.78%	37.36%	40.18%	1.5493

TABLE X: Fraction of cases where the slowdown prediction error is within the given error range.

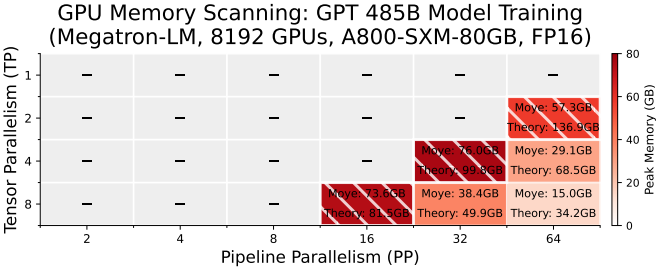

Fig. 16: GPU memory constraint scanning for a 485B model on 8192 GPUs. Each cell represents a unique parallelism strategy defined by its PP and TP sizes, with DP implicitly set to satisfy PP * TP * DP = 8192. Gray cells indicate OOM conditions. The red, hatched cells highlight configurations that are empirically feasible yet are incorrectly identified as OOM by baseline analytical models.

Model-wise evaluation. We further evaluate the impact of slowdown-aware modeling on training simulation accuracy across representative models. Experiments are conducted on modern training setups that already incorporate computation–communication overlapping: VGG19 and GPT2 with PyTorch DDP, GPT-6.7B with Megatron-LM 3D parallelism, and Mixtral-8×1.75B with Flux, which enables fine-grained GroupedGEMM overlap. We slightly adjust model parameters so that overlapped kernels account for more than 25% of execution time in all cases. Compared to baselines that ignore slowdown effects (FlexFlow and Calculon), incorporating slowdown improves the accuracy of simulating the computation portion by about 4% on average. At the model level, Moye achieves the lowest prediction error across all tested architectures: 2–5% on average, compared to 8–23% for Proteus and 10–37% for FlexFlow/Calculon (Figure 15). These results highlight that accurate slowdown prediction is essential for faithfully modeling modern training workloads with heavy overlap.

IX. Use Case Study

A. Use Case: Training Strategy Search

A recurring practical question in large-scale training is: given a GPU cluster and the LLM, how should the training job and the parallelisms be configured to achieve the best efficiency? Answering this question by trial-and-error on the real cluster is prohibitively expensive, as each trial may waste thousands of GPU-hours. Moye provides a low-cost alternative by efficiently searching across a wide range of parallelism and system configurations.

The workflow proceeds in two stages. First, Moye conducts a GPU memory feasibility scan via grid search over parallelism choices (see Figure 16). This step leverages its high-fidelity memory simulation to filter out infeasible configurations and avoid false OOM predictions that plague existing analytical models (e.g., Proteus, Calculon). Second, for all surviving candidates, Moye performs end-to-end simulation to estimate throughput and cost. By faithfully reconstructing the interaction between computation, communication, and memory behaviors, Moye enables accurate evaluation of training efficiency under each configuration. From these results, Moye recommends the configuration that minimizes time-to-train and monetary cost (see Table XI).

B. Use Case: Scaling-up Analysis

Larger TP does not improve E2E throughput. A natural expectation is that scaling up TP to match more GPUs within a node can exploit fast NVLink/NVSwitch links and accelerate model-shard execution. We test this intuition on GPT-175B with 1,024 GPUs, fixed PP=8, and global batch size 256. Figure 17(a) shows the opposite trend: normalized throughput drops from 1.0 at TP=8 to 0.57 and 0.32 at TP=16 and TP=32, corresponding to iteration time increasing from 34.4 s to 60.5 s and 106.2 s. Thus, simply enlarging the TP domain can hurt rather than help end-to-end efficiency.

Setting	Peak memory	Throughput	Time (days)	Cost
(PP16, TP8, DP64)	73.6 GB	455.8k tokens/s	12.7	$2.45M
(PP32, TP8, DP32)	38.4 GB	361.1k tokens/s	16.0	$3.09M
(PP64, TP8, DP16)	15.0 GB	331.9k tokens/s	17.4	$3.36M
(PP32, TP4, DP64)	76.0 GB	350.0k tokens/s	16.5	$3.19M
(PP64, TP4, DP32)	29.1 GB	222.1k tokens/s	26.1	$5.02M
(PP64, TP2, DP64)	57.3 GB	308.3k tokens/s	18.8	$3.62M

TABLE XI: Simulation results under different training configurations on 8,192 A800 GPUs for GPT 485B with a volume of 500B tokens. The training time is calculated using the total volume and predicted throughput, and the cost is derived accordingly using the market rent price of A800. The red row denotes Moye's choice, while the gray row is Calculon's choice which is suboptimal.

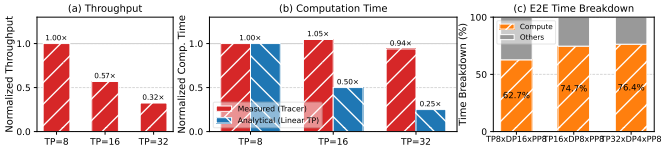

Fig. 17: Scaling-up analysis for GPT-175B on 1,024 GPUs (fixed PP=8 and global batch size 256). **(a)** Normalized throughput under different TP degrees. **(b)** Normalized per-microbatch computation time from Moye's framework-specific ex-situ tracer versus an analytical model assuming ideal linear TP scaling. **(c)** Stage0 time breakdown into computation and *others* across TP×DP×PP configurations.

Why it happens: TP increases, DP shrinks, but per-microbatch compute does not. Under fixed world size and PP, increasing TP compresses DP from 16 to 8 to 4, which increases the number of micro-batches per iteration from 16 to 32 to 64. This trade-off would be acceptable only if each micro-batch became proportionally cheaper. However, Figure 17(b) shows that Moye's tracer measures nearly flat per-microbatch computation time: 1,349.7 ms, 1,412.1 ms, and 1,267.2 ms for TP=8/16/32. In contrast, a naive analytical model assuming ideal linear TP scaling predicts 1,349.7 ms, 674.8 ms, and 337.4 ms. Because the expected compute speedup never materializes, total stage0 computation rises sharply from 21.6 s to 45.2 s and 81.1 s, which directly drives the throughput collapse.

Takeaway for scaling-up decisions. Figure 17(c) confirms that computation dominates the bottleneck stage across TP8×DP16×PP8, TP16×DP8×PP8, and TP32×DP4×PP8: its share grows from 62.7% to 74.7% and 76.4%, while *others* remain non-trivial. Therefore, once memory feasibility is satisfied, TP should be treated as a tunable parameter rather than maximized by default; when larger TP only reduces DP and increases micro-batching, additional scale-up can reduce throughput. This use case highlights the value of Moye's high-fidelity tracing inputs for exposing non-ideal scaling and guiding reliable LLM training configuration decisions.

X. Limitations and Discussions

Moye has limitations and we wish to improve it in at least the following directions.

Communication prediction. Moye positions itself between hand-tuned α–β models and packet-level network simulators. We plan to investigate learning based solutions like m3 [53] that efficiently predicts flow-level performance while capturing network-centric factors such as transport protocols.

Parallelism. Moye does not support sequence/context parallelism currently. We plan to integrate them in subsequent releases.

XI. Related Work

We discuss related work other than those covered in §II.

GPU simulation. GPU computation simulators like SCALE-sim [54] and Accel-Sim [55] focus on instruction-level modeling, while other approaches [15], [16], [19], [22] rely on profiling or tracing to capture accurate operation execution times. Moye similarly adopts a trace-based methodology, but crucially does so without requiring a full-scale deployment.

Network simulation. Discrete event simulators like ns-3 [27] and OMNeT++ [56] are well-known to have high overheads due to their packet-level whole-stack simulation. Other than parallelization optimizations [57], [58], recently learning based approach to predict packet-level or flow-level performance without simulating the whole stack [53], [59], [60] has shown promising results. As mentioned before, integrating them with Moye for training simulation is a promising direction for future work.

XII. Conclusion

We presented Moye, a high-fidelity simulator for large-scale distributed ML training that bridges critical gaps in existing approaches. Moye simulates without full-scale deployment using ex-situ workload tracing, estimates CC performance via white-box modeling, and accounts for the slowdown caused by kernel overlapping. Our evaluation shows that Moye not only achieves up to 3x lower simulation error than state-of-the-art baselines but also is highly efficient and practical. We plan to open source Moye to the community.

References

[1] Z. Jiang, H. Lin, Y. Zhong, Q. Huang, Y. Chen, Z. Zhang, Y. Peng, X. Li, C. Xie, S. Nong *et al.*, "{MegaScale}: Scaling large language model training to more than 10,000 {GPUs}," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 2024, pp. 745–760.

[2] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, "Accurate, large minibatch sgd: Training imagenet in 1 hour," *arXiv preprint arXiv:1706.02677*, 2017.

[3] https://ai.meta.com/blog/meta-llama-3-1, 2024.

[4] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are rnns: Fast autoregressive transformers with linear attention," in *International conference on machine learning*. PMLR, 2020, pp. 5156–5165.

[5] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *ICLR*, 2017.

[6] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer *et al.*, "Pytorch fsdp: experiences on scaling fully sharded data parallel," *arXiv preprint arXiv:2304.11277*, 2023.

[7] J. Ren, S. Rajbhandari, R. Y. Aminabadi, O. Ruwase, S. Yang, M. Zhang, D. Li, and Y. He, "{ZeRO-Offload}: Democratizing {Billion-Scale} model training," in *USENIX ATC*, 2021.

[8] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[9] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision," https://arxiv.org/abs/2407.08608, 2024.

[10] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.

[11] "MemryX MX3 Chip," https://memryx.com/products/, 2024.

[12] "Simai: Unifying architecture design and performance tunning for large-scale large language model training with scalability and precision," in *22th USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, 2025.

[13] J. Duan, X. Li, P. Xu, X. Zhang, S. Yan, Y. Liang, and D. Lin, "Proteus: Simulating the performance of distributed dnn training," *IEEE Transactions on Parallel and Distributed Systems*, 2024.

[14] W. Won, T. Heo, S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "Astra-sim2. 0: Modeling hierarchical networks and disaggregated systems for large-model training at scale," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2023, pp. 283–294.

[15] G. Lu, R. Chen, Y. Wang, Y. Zhou, R. Zhang, Z. Hu, Y. Miao, Z. Cai, L. Li, J. Leng *et al.*, "Distsim: A performance model of large-scale hybrid distributed dnn training," in *Proceedings of the 20th ACM International Conference on Computing Frontiers*, 2023, pp. 112–122.

[16] H. Hu, C. Jiang, Y. Zhong, Y. Peng, C. Wu, Y. Zhu, H. Lin, and C. Guo, "dpro: A generic performance diagnosis and optimization toolkit for expediting distributed dnn training," *Proc. MLSys*, 2022.

[17] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.

[18] Z. Jia, M. Zaharia, and A. Aiken, "Beyond data and model parallelism for deep neural networks." *Proceedings of Machine Learning and Systems*, vol. 1, pp. 1–13, 2019.

[19] H. Zhu, A. Phanishayee, and G. Pekhimenko, "Daydream: Accurately estimating the efficacy of optimizations for {DNN} training," in *USENIX ATC*, 2020.

[20] M. Isaev, N. Mcdonald, L. Dennison, and R. Vuduc, "Calculon: a methodology and tool for high-level co-design of systems and large language models," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–14.

[21] M. Liang, H. T. Kassa, W. Fu, B. Coutinho, L. Feng, and C. Delimitrou, "Lumos: Efficient performance modeling and estimation for large-scale llm training," *arXiv preprint arXiv:2504.09307*, 2025.

[22] L. Luo, P. West, P. Patel, A. Krishnamurthy, and L. Ceze, "Srifty: Swift and thrifty distributed neural network training on the cloud," *Proceedings of Machine Learning and Systems*, vol. 4, pp. 833–847, 2022.

[23] J. Duan, S. Zhang, Z. Wang, L. Jiang, W. Qu, Q. Hu, G. Wang, Q. Weng, H. Yan, X. Zhang *et al.*, "Efficient training of large language models on distributed infrastructures: A survey," *arXiv preprint arXiv:2407.20018*, 2024.

[24] P. Qi, X. Wan, G. Huang, and M. Lin, "Zero bubble pipeline parallelism," *arXiv preprint arXiv:2401.10241*, 2023.

[25] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu *et al.*, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," *Proc. NeurIPS*, 2019.

[26] L. G. Valiant, "A bridging model for parallel computation," *Communications of the ACM*, vol. 33, no. 8, pp. 103–111, 1990.

[27] G. F. Riley and T. R. Henderson, "The ns-3 network simulator," in *Modeling and tools for network simulation*. Springer, 2010, pp. 15–34.

[28] L.-W. Chang, W. Bao, Q. Hou, C. Jiang, N. Zheng, Y. Zhong, X. Zhang, Z. Song, C. Yao, Z. Jiang *et al.*, "Flux: Fast software-based communication overlap on gpus through kernel fusion," *arXiv preprint arXiv:2406.06858*, 2024.

[29] C. Chen, X. Li, Q. Zhu, J. Duan, P. Sun, X. Zhang, and C. Yang, "Centauri: Enabling efficient scheduling for communication-computation overlap in large model training via communication partitioning," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 178–191.

[30] H. Zhang, Z. Zheng, S. Xu, W. Dai, Q. Ho, X. Liang, Z. Hu, J. Wei, P. Xie, and E. P. Xing, "Poseidon: An efficient communication architecture for distributed deep learning on {GPU} clusters," in *USENIX ATC*, 2017.

[31] C. Hwang, K. Park, R. Shu, X. Qu, P. Cheng, and Y. Xiong, "{ARK}:{GPU-driven} code execution for distributed deep learning," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 87–101.

[32] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 3505–3506.

[33] A. Weingram, Y. Li, H. Qi, D. Ng, L. Dai, and X. Lu, "xccl: A survey of industry-led collective communication libraries for deep learning," *Journal of Computer Science and Technology*, vol. 38, no. 1, pp. 166–195, 2023.

[34] Y. Li, Y. Bao, G. Wang, X. Mei, P. Vaid, A. Ghosh, A. Jog, D. Bunandar, A. Joshi, and Y. Sun, "Triosim: A lightweight simulator for large-scale dnn workloads on multi-gpu systems," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1524–1538.

[35] J. Bang, Y. Choi, M. Kim, Y. Kim, and M. Rhu, "vtrain: A simulation framework for evaluating cost-effective and compute-optimal large language model training," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 153–167.

[36] "Nvidia Collective Communications Library (NCCL)," https://developer.nvidia.com/nccl, 2024.

[37] "Developing a Linux Kernel Module using GPUDirect RDMA," https://docs.nvidia.com/cuda/gpudirect-rdma/index.html, 2022.

[38] "GDRCopy," https://github.com/NVIDIA/gdrcopy, 2022.

[39] "A fast communication-overlapping library for tensor/expert parallelism on GPUs." https://github.com/bytedance/flux, 2025.

[40] "Massively Scale Your Deep Learning Training with NCCL 2.4," https://developer.nvidia.com/blog/massively-scale-deep-learning-training-nccl-2-4/, 2019.

[41] "NCCL GPU-side implementation," https://github.com/NVIDIA/nccl/tree/master/src/device, 2025.

[42] Z. Hu, S. Shen, T. Bonato, S. Jeaugey, C. Alexander, E. Spada, J. Dinan, J. Hammond, and T. Hoefler, "Demystifying nccl: An in-depth analysis of gpu communication protocols and algorithms," *arXiv preprint arXiv:2507.04786*, 2025.

[43] "NCCL ring and tree algo src codes," https://github.com/NVIDIA/nccl/tree/master/src/graph, 2025.

[44] "XGBoost," https://xgboost.readthedocs.io/en/stable/, 2014.

[45] "NPKit: NCCL Profiling Kit," https://github.com/microsoft/NPKit/tree/main, 2023.

[46] "An interactive profiler for CUDA and NVIDIA OptiX," https://developer.nvidia.com/nsight-compute, 2025.

[47] "A system-wide performance analysis tool," https://developer.nvidia.com/nsight-systems, 2025.

[48] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

[49] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[50] "qwen3-a3b," https://huggingface.co/Qwen/Qwen3-30B-A3B.

[51] "deepseek-v3," https://huggingface.co/deepseek-ai/DeepSeek-V3.

[52] DeepSeek, "DeepEP," https://github.com/deepseek-ai/DeepEP.

[53] C. Li, A. Nasr-Esfahany, K. Zhao, K. Noorbakhsh, P. Goyal, M. Alizadeh, and T. E. Anderson, "m3: Accurate flow-level performance estimation using machine learning," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 813–827.

[54] A. Samajdar, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "Scale-sim: Systolic cnn accelerator simulator," *arXiv preprint arXiv:1811.02883*, 2018.

[55] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-sim: An extensible simulation framework for validated gpu modeling," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 473–486.

[56] A. Varga, "A practical introduction to the omnet++ simulation framework," in *Recent advances in network simulation: the OMNeT++ environment and its ecosystem*. Springer, 2019, pp. 3–51.

[57] K. Gao, L. Chen, D. Li, V. Liu, X. Wang, R. Zhang, and L. Lu, "DONS: Fast and Affordable Discrete Event Network Simulation with Automatic Parallelization," in *Proc. ACM SIGCOMM*, 2023.

[58] S. Bai, H. Zheng, C. Tian, X. Wang, C. Liu, X. Jin, F. Xiao, Q. Xiang, W. Dou, and G. Chen, "Unison: A Parallel-Efficient and User-Transparent Network Simulation Kernel," in *Proc. ACM EuroSys*, 2024.

[59] Q. Zhang, K. K. W. Ng, C. Kazer, S. Yan, J. a. Sedoc, and V. Liu, "MimicNet: Fast Performance Estimates for Data Center Networks with Machine Learning," in *Proc. ACM SIGCOMM*, 2021.

[60] Q. Yang, X. Peng, L. Chen, L. Liu, J. Zhang, H. Xu, B. Li, and G. Zhang, "DeepQueueNet: Towards Scalable and Generalized Network Performance Estimation with Packet-Level Visibility," in *Proc. ACM SIGCOMM*, 2022.